Yes — **this is a genuinely strong project idea**, especially because it solves a real interview problem rather than being "just another code editor."

The core idea would be:

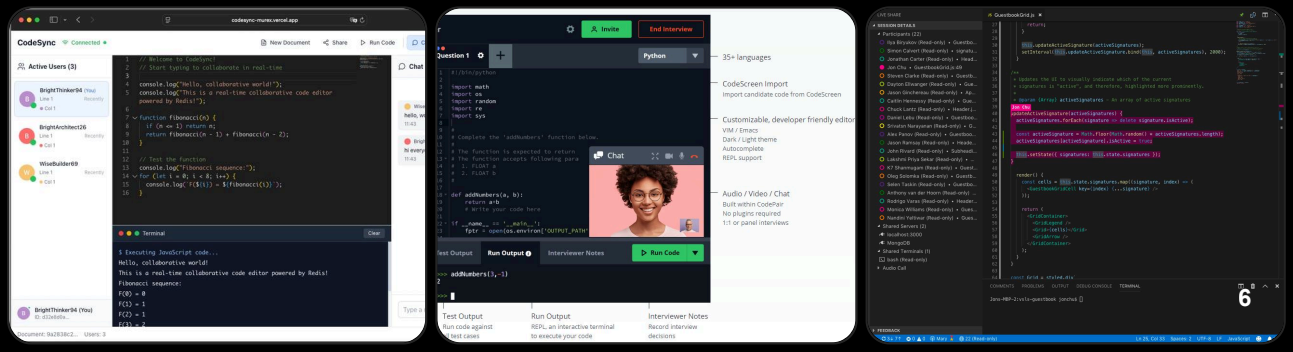
> **Create a shared coding workspace where an interviewer and candidate can simultaneously edit the same code, communicate, run code, and optionally share test cases — without copying/pasting interview questions or code.**

Think of it as a lightweight combination of **Google Docs + VS Code + an online compiler**, specifically designed for technical interviews.

I'd build it in stages rather than trying to make everything at once.

---

1. What your final product could look like



A user opens your website and gets something like:

```
┌──────────────────────────────────────────────────────────────┐
│ CodeTogether                Room: X7K29     👤 Arunabh 👤 A  │
├──────────────┬──────────────────────────────┬────────────────┤
│              │                              │                │
│  Problem     │        Code Editor           │   Chat         │
│              │                              │                │
│  Two Sum     │  int twoSum(...) {           │ Interviewer:   │
│              │      ...                     │ "Can you       │
│  Given an    │  }                           │ explain this?" │
│  array...    │                              │                │
│              │                              │ Candidate:     │
│              │                              │ "Sure..."      │
│              │                              │                │
├──────────────┴──────────────────────────────┴────────────────┤
│ Language: C++      ▶ Run      Test Cases      Output         │
└──────────────────────────────────────────────────────────────┘
```

And the killer feature:

Both people can type simultaneously.

If the interviewer types:

```cpp
int solve(vector<int>& nums)
```

the candidate immediately sees it.

If the candidate changes:

```cpp
int solve(vector<int>& nums, int target)
```

the interviewer immediately sees that too.

No refreshing.

No copying.

No sending screenshots.

---

2. The most important technical problem

The difficult part isn't actually making a code editor.

You can embed a sophisticated editor relatively easily.

The **hard part is synchronizing two people's edits in real time.**

For example:

Initially:

```
abc
```

Candidate types `X`:

```
aXbc
```

At almost the same time interviewer types `Y`:

```
abYc
```

Your server needs to figure out how the final document should become:

```
aXbYc
```

rather than:

```
abYc
```

or:

```
aXbc
```

This is where you need to understand:

CRDTs

and/or

Operational Transformation (OT)

For your project, I'd strongly recommend **CRDT** rather than implementing synchronization algorithms yourself.

---

3. Recommended tech stack

If you're comfortable with JavaScript and C++, I'd use:

Part	Technology
Frontend	React + TypeScript
Code editor	Monaco Editor
Real-time synchronization	Yjs
WebSocket layer	Node.js + Socket.IO/WebSocket
Backend	Node.js + Express
Database	PostgreSQL
Fast temporary data	Redis
Authentication	JWT / OAuth

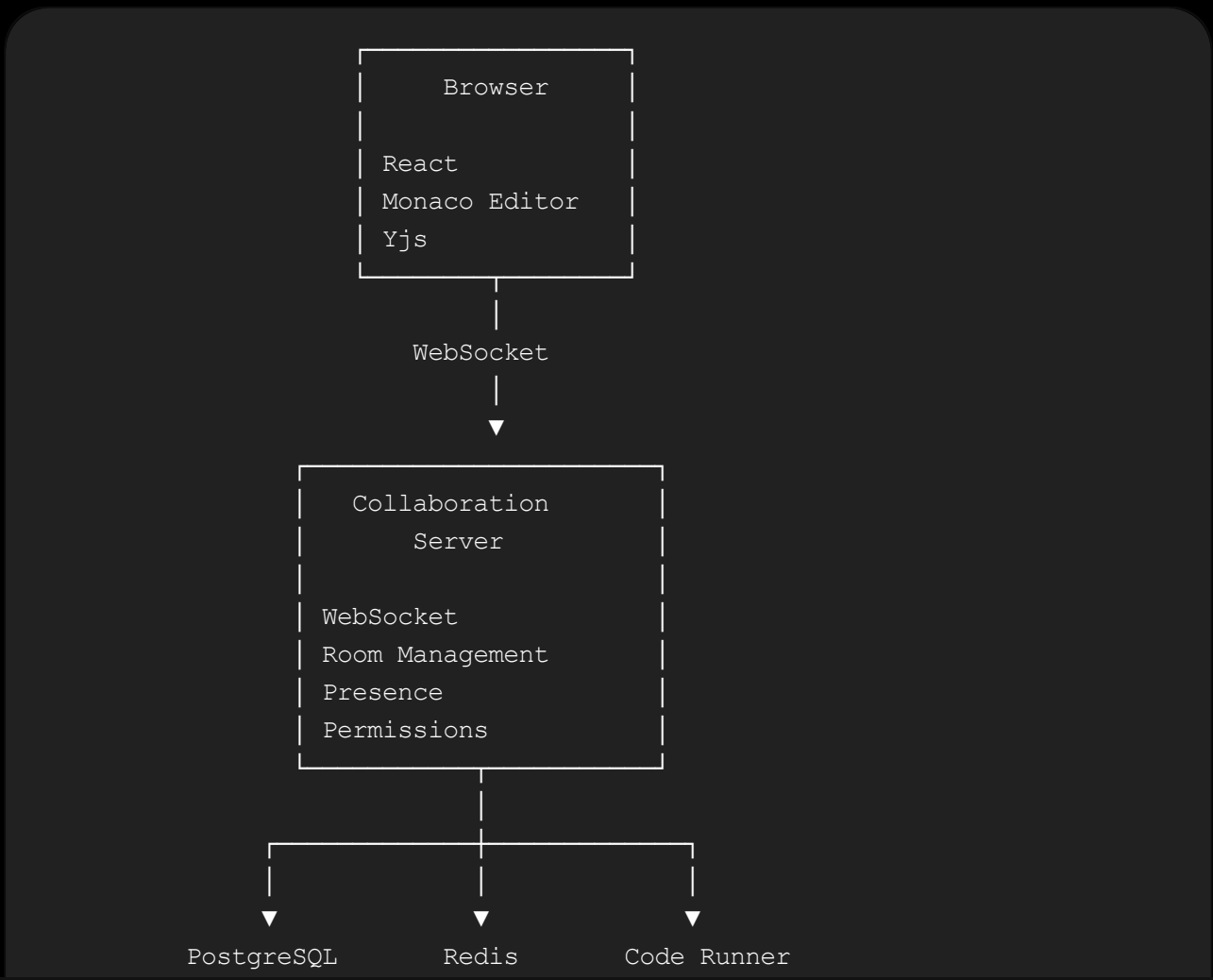
Part	Technology
Code execution	Docker containers
Styling	Tailwind CSS
API	REST
Real-time API	WebSockets
Deployment	Vercel + Railway/Render/AWS
Reverse proxy	Nginx
Version control	Git + GitHub

You don't need all of these for V1.

---

4. Architecture

Your architecture could eventually look like this:



```
Users/Rooms     Sessions       Docker
Problems        Presence       Sandbox
History
```

---

5. Break the project into systems

Don't think of this as one project.

Think of it as roughly **10 smaller systems**.

1. Authentication

Users need accounts.

2. Room system

Create/join interview rooms.

3. Collaborative editor

Multiple users edit simultaneously.

4. Presence

See who is online.

5. Communication

Chat between interviewer and candidate.

6. Problem system

Display coding questions.

7. Code execution

Run candidate's code.

8. Session history

Save what happened during the interview.

9. Permissions

Interviewer and candidate shouldn't necessarily have identical powers.

10. Deployment/security

Make the whole thing usable on the internet.

---

6. Start with the MVP

Do **not** start with authentication, Docker, AI, video calls, etc.

Your first version should literally be:

with:

```
Monaco Editor
      +
WebSocket
      +
shared document
```

That's it.

Your first milestone:

> **Open two browser tabs and type in one. See the changes instantly in the other.**

If you can achieve that, you've solved the most important technical challenge.

---

7. Monaco Editor

This is the editor used by VS Code.

You don't need to build:

- syntax highlighting
- autocomplete
- bracket matching
- minimap
- line numbers
- error highlighting
- keyboard shortcuts

from scratch.

Monaco already gives you these.

You can configure:

```
Language:
C
C++
Java
Python
JavaScript
TypeScript
Go
Rust
...
```

For example:

```
JavaScript

<Editor
    height="600px"
    defaultLanguage="cpp"
    theme="vs-dark"
/>
```

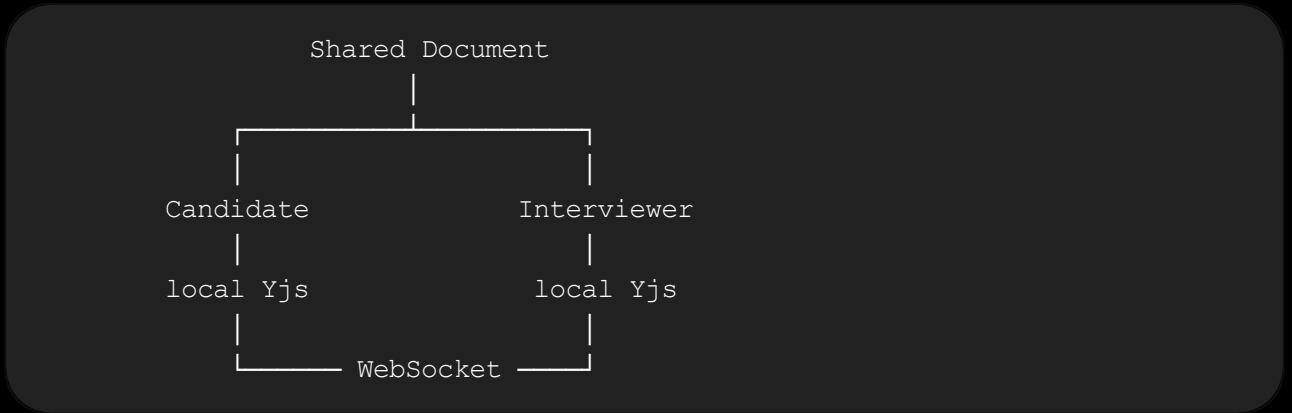
Your job is to synchronize the document behind the editor.

---

8. Yjs — probably the most important technology for your project

Yjs is a CRDT framework.

Conceptually:



Both clients have a local representation of the document.

Changes are synchronized.

So you don't have to invent:

```
"if user A types here while user B types there..."
```

yourself.

---

9. What exactly should be synchronized?

Not just the text.

You can synchronize:

Code

```
C++
int main() {
}
```

Cursor position

```
Arunabh's cursor → line 7
Interviewer cursor → line 12
```

Selection

You could show:

```
Candidate selected:
vector<int> nums
```

Presence

```
🟢 Arunabh
🟢 Interviewer
```

Language

```
C++
```

Problem

```
Two Sum
```

Possibly terminal/output state

Although I would initially keep execution state separate.

---

10. Presence system

This is a surprisingly nice feature.

Suppose the interviewer is currently on:

```
   C++

for(int i = 0; i < n; i++)
```

You could display:

```
        |
        | ← Interviewer
        |
```

with a colored cursor/name.

Something like:

```
👤 Interviewer
   Line 14
```

This makes the application feel **actually collaborative** rather than simply "two people sharing a text box."

---

11. Rooms

Every interview should have a room.

For example:

```
https://yourapp.com/room/X7K29
```

The room ID:

```
X7K29
```

identifies the session.

Database:

```
rooms
----------------
id
created_by
created_at
status
language
problem_id
```

Users join:

```
POST /rooms
```

and receive:

```
    JSON

{
    "roomId": "X7K29"
}
```

Then:

```
/interview/X7K29
```

opens the workspace.

---

12. Interviewer vs candidate

This is where your application can become much more interesting.

Give different roles.

Interviewer

Can:

- create room
- select problem
- edit problem

- edit code
- run code
- reset workspace
- view candidate activity
- end interview

Candidate

Can:

- edit code
- run code
- view problem
- chat
- submit solution

Potentially:

```
Interviewer
      |
      ├── Problem
      ├── Code
      ├── Tests
      ├── Chat
      └── Session controls

Candidate
      |
      ├── Problem
      ├── Code
      ├── Tests
      └── Chat
```

---

13. The interview problem system

You need a database of questions.

For example:

```
Problem
------------------------
id
title
description
difficulty
constraints
```

```
input_format
output_format
examples
starter_code
test_cases
tags
```

Example:

```
ID: 42
Title: Two Sum
Difficulty: Easy
Language: C++
```

The interviewer chooses:

```
Two Sum
```

and both participants immediately see it.

---

14. Code execution

This is the **second hardest part**.

Suppose the candidate submits:

```
C++

#include <iostream>

int main() {
    while(true) {}
}
```

You absolutely **cannot execute that directly on your server**.

Otherwise somebody could:

```
delete files
consume CPU
consume RAM
open network connections
spawn processes
attack the server
```

You need a sandbox.

---

15. Docker-based code runner

Conceptually:

```
Candidate Code
      |
      ▼
Backend
      |
      ▼
Create isolated container
      |
      ├── CPU limit
      ├── RAM limit
      ├── execution timeout
      ├── no network
      └── restricted filesystem
      |
      ▼
Compile
      |
      ▼
Run
      |
      ▼
Capture stdout/stderr
      |
      ▼
Return result
```

For example:

```
POST /execute
```

Request:

JSON

```
{
    "language": "cpp",
    "code": "...",
    "stdin": "5\n1 2 3 4 5"
}
```

Response:

JSON

```
{
```

```
    "stdout": "15",
    "stderr": "",
    "executionTime": 0.13,
    "status": "success"
}
```

---

16. Don't initially build your own compiler infrastructure

For your first version, you have two choices.

Option A — external code execution API

Much easier.

Your server sends:

```
code
language
stdin
```

to a code execution service.

Option B — your own Docker runner

Much more impressive as a project.

Eventually:

```
Node Backend
      ↓
Job Queue
      ↓
Runner
      ↓
Docker Container
      ↓
Result
```

I would eventually choose B if you're building this as a serious portfolio project.

---

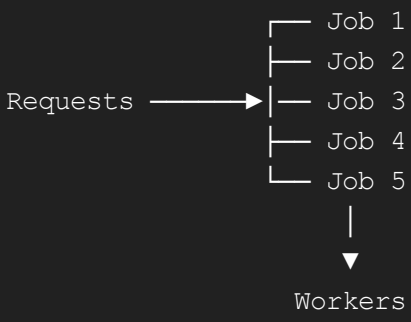
17. Job queue

Imagine 500 interviews happen simultaneously.

You don't want:

```
500 users
   ↓
500 containers
   ↓
server explodes 💀
```

Instead:



Redis + BullMQ is a common architecture for this.

---

18. Chat

Add a small interview chat.

WebSockets make this easy because you already have real-time infrastructure.

---

19. Interview timer

Very useful.

```
Interview duration
```

```
01:27:34
```

Or:

```
┌──────────────────┐
│ Time Remaining   │
│    34:21         │
└──────────────────┘
```

The interviewer sets:

```
Duration = 60 minutes
```

Both clients receive the same session start time.

Don't synchronize a constantly ticking timer through WebSockets.

Store:

```
startTime
duration
```

and let each browser calculate:

```
remaining = startTime + duration - currentTime
```

---

20. Interview recording/history

This could become one of your strongest features.

After the interview:

```
Interview Summary
_____________________

Candidate:
Arunabh

Problem:
Two Sum

Duration:
47 minutes
```

```
Languages:
C++

Final Code:
...

Tests Passed:
8/10

Interviewer Notes:
...

Timeline:
00:03 Candidate started coding
00:17 First solution
00:29 Optimized solution
00:42 All tests passed
```

You could potentially replay the coding session.

---

21. Code version history

Imagine:

```
Version 1
↓
Version 2
↓
Version 3
↓
Final
```

Interviewer can see how the candidate arrived at the answer.

This is actually **very valuable for interviews** because you don't just care about the final code.

You care about:

> How did they think?

For example:

```
10:02
Brute force solution

10:15
Recognized O(n²)
```

```
10:20
Started using hashmap

10:27
Final O(n) solution
```

That's a killer feature.

---

22. Undo/redo becomes complicated

With collaborative editing:

```
Candidate types A
Interviewer types B
Candidate presses Ctrl+Z
```

What exactly should happen?

You don't want candidate's undo to erase the interviewer's change.

Again, this is one reason to use a mature CRDT solution rather than implementing collaboration yourself.

---

23. Authentication

Eventually:

```
Sign up
   ↓
Login
   ↓
Dashboard
```

Dashboard:

```
My Interviews

┌──────────────────────────────────┐
│ Two Sum          Aug 14          │
│ Candidate: XYZ                   │
│ Completed                        │
├──────────────────────────────────┤
│ Binary Tree     Aug 13           │
│ Candidate: ABC                   │
```

```
| Completed                          |
└────────────────────────────────────┘
```

You could support:

```
Google login
Email/password
GitHub login
```

But don't build this first.

---

24. Database design

You could have:

users

```
id
name
email
password_hash
created_at
```

rooms

```
id
interviewer_id
candidate_id
problem_id
status
created_at
started_at
ended_at
```

problems

```
id
title
description
difficulty
starter_code
```

submissions

```
id
```

```
room_id
user_id
code
language
status
execution_time
created_at
```

messages

```
id
room_id
user_id
message
created_at
```

interview_notes

```
id
room_id
interviewer_id
notes
```

---

25. Redis

PostgreSQL shouldn't be responsible for everything.

Redis can handle temporary/high-speed information:

```
room:X7K29
```

containing:

```
connected users
active session
presence
WebSocket information
```

You can also use it for:

```
rate limiting
job queues
temporary session data
pub/sub
```

26. Scaling WebSockets

Suppose you have:

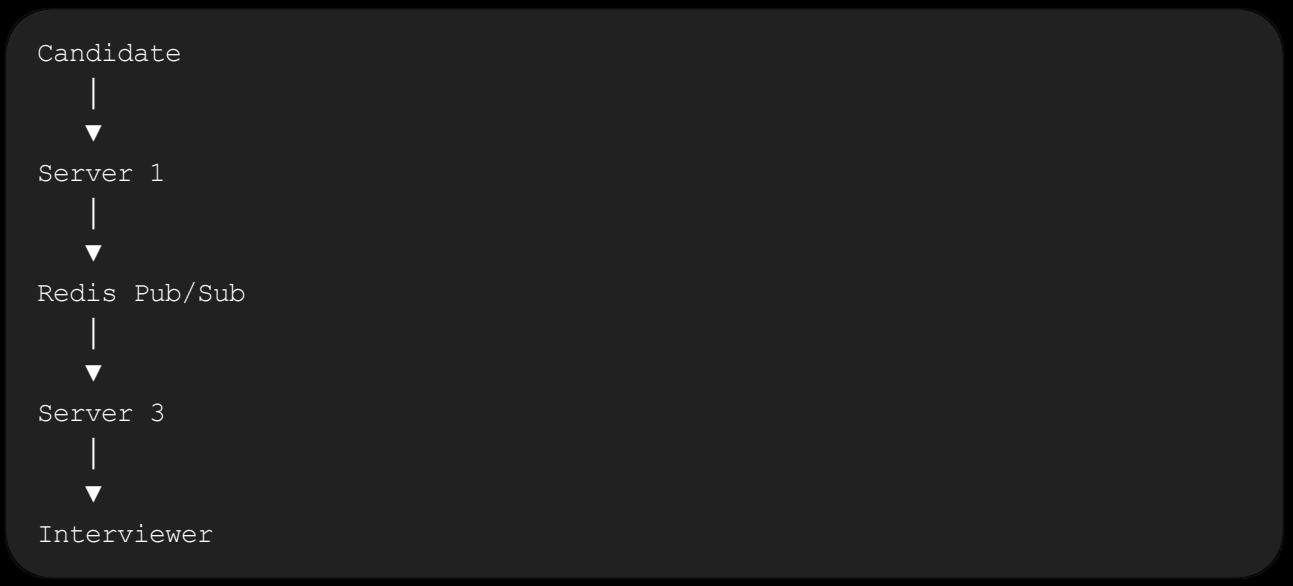
```
Server 1
Server 2
Server 3
```

Candidate connects to Server 1.

Interviewer connects to Server 3.

How does Server 1 know what happened on Server 3?

Redis Pub/Sub or another shared messaging layer.



You won't need this in your MVP, but understanding it is excellent interview material.

---

27. WebSocket vs WebRTC

You might wonder:

Why not WebRTC?

WebRTC is great for:

- video
- audio
- peer-to-peer data

But for your editor, I'd initially use:

```
WebSocket
```

because you want:

```
Client
  ↓
Server
  ↓
Client
```

The server can maintain room state, permissions, authentication, persistence, etc.

Later you could add:

```
Video call
Audio call
Screen sharing
```

using WebRTC.

---

28. Video interview feature

Eventually your interface could become:

Then you're basically building an **interview operating system**.

---

29. Voice/video isn't necessary for V1

Seriously.

Don't start with:

```
Authentication
Video
```

```
AI
Docker
Payments
Analytics
```

You'll never finish.

Start:

```
Monaco
+
Yjs
+
WebSocket
+
Rooms
```

Then progressively add features.

---

30. AI features

This is where you could make it particularly interesting later.

AI interviewer

The interviewer could optionally have an AI assistant.

For example:

```
Candidate:
"I'll use a hashmap."

AI Interviewer:
"Why is a hashmap preferable to sorting here?"
```

But be careful: this changes your project from a collaborative editor into an AI interviewing platform.

I'd make AI an **optional module**, not the core.

---

31. AI interviewer analytics

After the interview:

```
AI Analysis
```

```
Problem solving: 8/10
Communication: 7/10
Code quality: 9/10
Optimization: 8/10

Strengths:
• Identified hashmap approach quickly
• Good complexity analysis

Potential concerns:
• Initially missed duplicate case
```

That could become a major differentiator.

---

32. Security — extremely important

Because you're executing arbitrary code, security is not optional.

You need:

Authentication

Don't trust client-provided user IDs.

Authorization

Candidate shouldn't be able to:

```
DELETE ROOM
CHANGE INTERVIEWER
VIEW PRIVATE NOTES
```

Rate limiting

Prevent:

```
POST /execute
POST /execute
POST /execute
...
```

Code execution sandbox

Restrict:

```
CPU
```

```
RAM
execution time
filesystem
network
process count
```

Input limits

Don't allow:

```
500 MB source code
10 GB stdin
```

WebSocket authentication

Don't simply accept:

```
ws://server/room/X7K29
```

and assume the user belongs there.

---

33. XSS protection

Chat is another potential attack vector.

Someone sends:

```HTML
<script>
  stealSomething()
</script>
```

Your application should never blindly inject user-generated HTML.

React helps, but you still need proper server-side validation and safe rendering.

---

34. SQL injection

Never construct:

```JavaScript
query = "SELECT * FROM users WHERE name = '" + username + "'";
```

Use:

```
parameterized queries
```

or an ORM/query builder.

---

35. HTTPS/WSS

Production should use:

```
HTTPS
```

and:

```
WSS
```

rather than:

```
HTTP
WS
```

especially because interview data and code may be private.

---

36. Project folder structure

I'd structure your backend roughly like:

```
backend/
│
├── src/
│   ├── controllers/
│   │   ├── auth.controller.js
│   │   ├── room.controller.js
│   │   └── problem.controller.js
│   │
│   ├── routes/
│   │   ├── auth.routes.js
│   │   ├── room.routes.js
│   │   └── problem.routes.js
│   │
│   ├── websocket/
│   │   ├── room.socket.js
│   │   └── presence.socket.js
│   │
```

```
│   ├── services/
│   │   ├── room.service.js
│   │   ├── execution.service.js
│   │   └── collaboration.service.js
│   │
│   ├── models/
│   │
│   ├── middleware/
│   │
│   └── server.js
│
├── Dockerfile
└── package.json
```

Frontend:

```
frontend/
│
├── src/
│   ├── components/
│   │   ├── Editor/
│   │   ├── Chat/
│   │   ├── Problem/
│   │   ├── Terminal/
│   │   └── Participants/
│   │
│   ├── pages/
│   │   ├── Login/
│   │   ├── Dashboard/
│   │   └── InterviewRoom/
│   │
│   ├── hooks/
│   │
│   ├── services/
│   │
│   └── App.tsx
```

---

37. API design

REST:

```
POST   /api/auth/register
POST   /api/auth/login

POST   /api/rooms
GET    /api/rooms/:id
DELETE /api/rooms/:id
```

```
GET    /api/problems
GET    /api/problems/:id

POST   /api/submissions
GET    /api/submissions/:id
```

WebSocket:

```
connect
join-room
leave-room
code-update
cursor-update
selection-update
chat-message
run-code
user-joined
user-left
```

---

38. State management

React state might become messy.

You could use:

```
Zustand
```

for client-side application state.

For example:

JavaScript

```
{
    roomId,
    user,
    role,
    language,
    problem,
    participants,
    interviewStatus
}
```

But don't over-engineer this either.

---

39. Testing

You'll need multiple types.

Unit tests

Test:

```
room creation
permissions
problem retrieval
```

Integration tests

Test:

```
login → create room → join room
```

Collaboration tests

Very important:

```
User A edits
User B edits
Both edits survive
```

Security tests

Try:

```
unauthorized room access
malicious code
oversized input
rate-limit abuse
```

Load testing

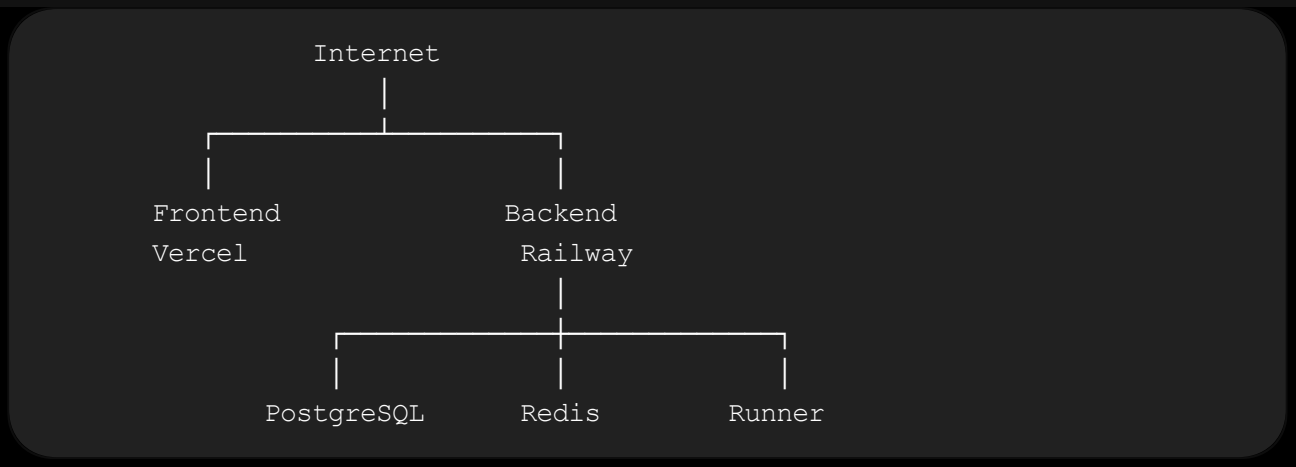
Eventually test:

```
100 rooms
200 WebSocket connections
```

---

40. Deployment

A simple deployment:



Later:

```
AWS
├── CloudFront
├── Load Balancer
├── ECS/Kubernetes
├── RDS
├── ElastiCache
└── EC2/Fargate runners
```

Don't start with AWS infrastructure unless you specifically want to learn DevOps.

---

41. Domain

Eventually something like:

```
codetogether.dev
```

or

```
paircode.dev
```

or

```
interviewcode.dev
```

Your room:

```
codetogether.dev/interview/X7K29
```

---

42. The feature roadmap I'd recommend

Phase 1 — Basic editor

Build:

```
React
+
Monaco
```

Features:

- editor
- language selection
- dark/light theme
- run button placeholder

Phase 2 — Real-time collaboration

Add:

```
WebSocket
+
Yjs
```

Build:

- rooms
- shared code
- multiple users
- live cursor
- presence

Goal:

Two laptops can open the same room and collaborate.

Phase 3 — Interview functionality

Add:

- interviewer
- candidate
- problem statement
- timer
- chat

- room controls

Now you have a **usable interview platform**.

Phase 4 — Code execution

Add:

```
Docker
+
execution service
```

Support:

```
C++
Python
Java
JavaScript
```

Start with **one language**, preferably C++ if that's what you know best.

Phase 5 — Persistence

Add:

```
PostgreSQL
```

Save:

- users
- rooms
- problems
- submissions
- interview history
- messages

Phase 6 — Authentication

Add:

```
JWT
```

or OAuth.

Then:

```
Dashboard
```

Phase 7 — Interview analytics

Add:

```
session timeline
code history
submissions
test results
interviewer notes
```

This makes the project substantially more impressive.

Phase 8 — Video/audio

Add:

```
WebRTC
```

Now the entire interview happens inside your application.

Phase 9 — AI

Finally:

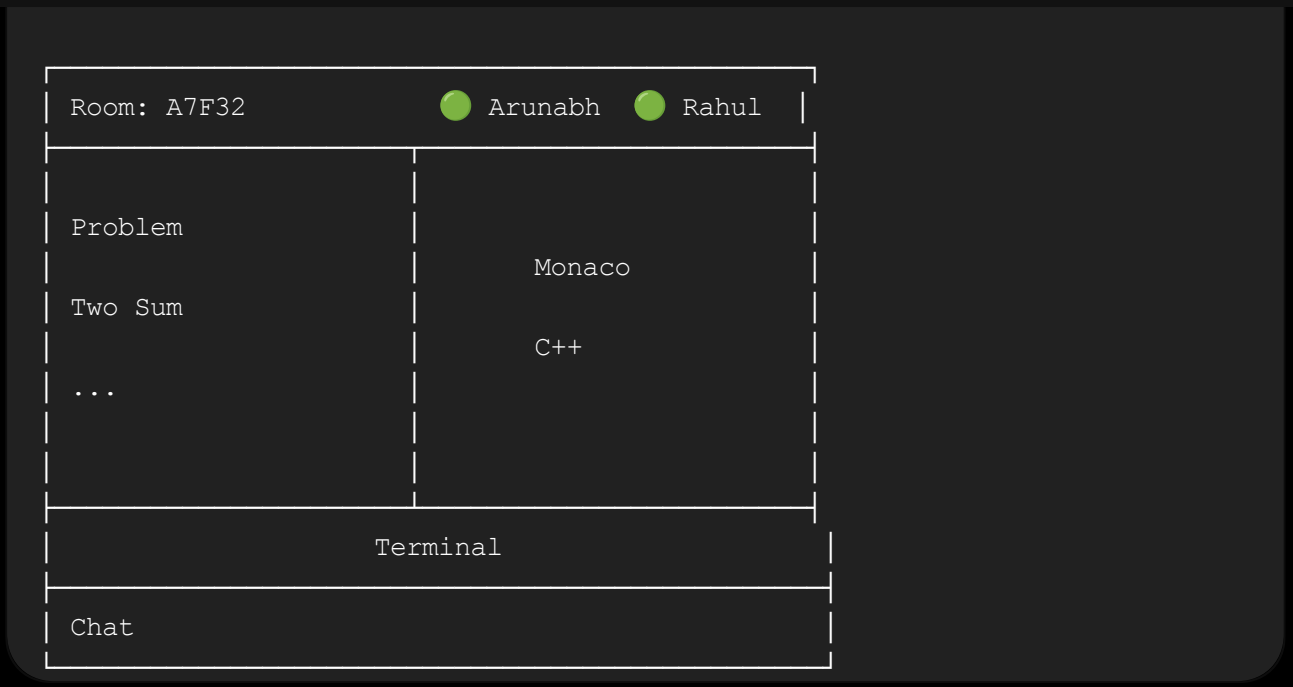
```
AI interviewer
AI code analysis
AI interview summary
```

---

43. Your MVP should actually be this small

If I were building it with you, I'd make **Version 1**:

```
                CODETOGETHER
```



Features:

Must have

- [ ] Create room
- [ ] Join room
- [ ] Monaco editor
- [ ] Real-time code synchronization
- [ ] Multiple users
- [ ] Live cursors
- [ ] Presence
- [ ] Problem statement
- [ ] Language selection
- [ ] Run code
- [ ] Chat
- [ ] Timer

Later

- [ ] Authentication
- [ ] Database
- [ ] Interview history
- [ ] Code replay
- [ ] Video
- [ ] AI interviewer
- [ ] Analytics

---

44. The technologies you should learn

If you want this project to also teach you a lot, I'd learn them in this order:

Level 1

```
HTML
CSS
JavaScript
React
TypeScript
```

Level 2

```
Node.js
Express
REST APIs
WebSockets
```

Level 3

```
Monaco Editor
Yjs
CRDT concepts
```

Level 4

```
PostgreSQL
Redis
Authentication
```

Level 5

```
Docker
Linux
process isolation
code execution
```

Level 6

```
AWS / cloud deployment
load balancing
WebSocket scaling
```

Level 7

```
WebRTC
AI APIs
observability
distributed systems
```

---

45. What will make this project stand out on your resume

Don't describe it as:

> "Made a collaborative code editor."

That's underselling it.

You could eventually say:

> **Built a real-time collaborative coding platform for technical interviews using React, Monaco Editor, WebSockets and CRDT-based synchronization, enabling multiple users to concurrently edit code with live cursor/presence updates.**

And if you implement the runner:

> **Designed an isolated Docker-based code execution service with resource limits and asynchronous job processing for secure multi-language code execution.**

And if you implement scaling:

> **Implemented Redis-backed WebSocket synchronization and session management to support distributed collaboration across multiple backend instances.**

Those are **excellent software engineering talking points** in an interview.

---

46. The really interesting CS concepts hidden inside this project

This project can teach you way more than just React.

You'll encounter:

Data structures

```
maps
sets
queues
trees
```

Networking

```
TCP
WebSockets
HTTP
WebRTC
```

Distributed systems

```
consistency
concurrency
replication
pub/sub
```

Databases

```
transactions
indexes
relationships
normalization
```

Operating systems

```
processes
containers
resource limits
sandboxing
```

Security

```
authentication
authorization
isolation
rate limiting
XSS
SQL injection
```

Algorithms

Most importantly:

```
CRDT
Operational Transformation
conflict resolution
```

So this isn't just a frontend project.

---

47. One thing I'd specifically recommend

Do not try to implement CRDT yourself initially.

Use:

```
Monaco
   +
Yjs
   +
y-websocket
```

and first understand **why** it works.

Then, once your application works, you can study:

```
What is a CRDT?
        ↓
Why do concurrent edits conflict?
        ↓
How are operations represented?
        ↓
How does Yjs resolve conflicts?
        ↓
How does offline editing work?
```

That gives you both:

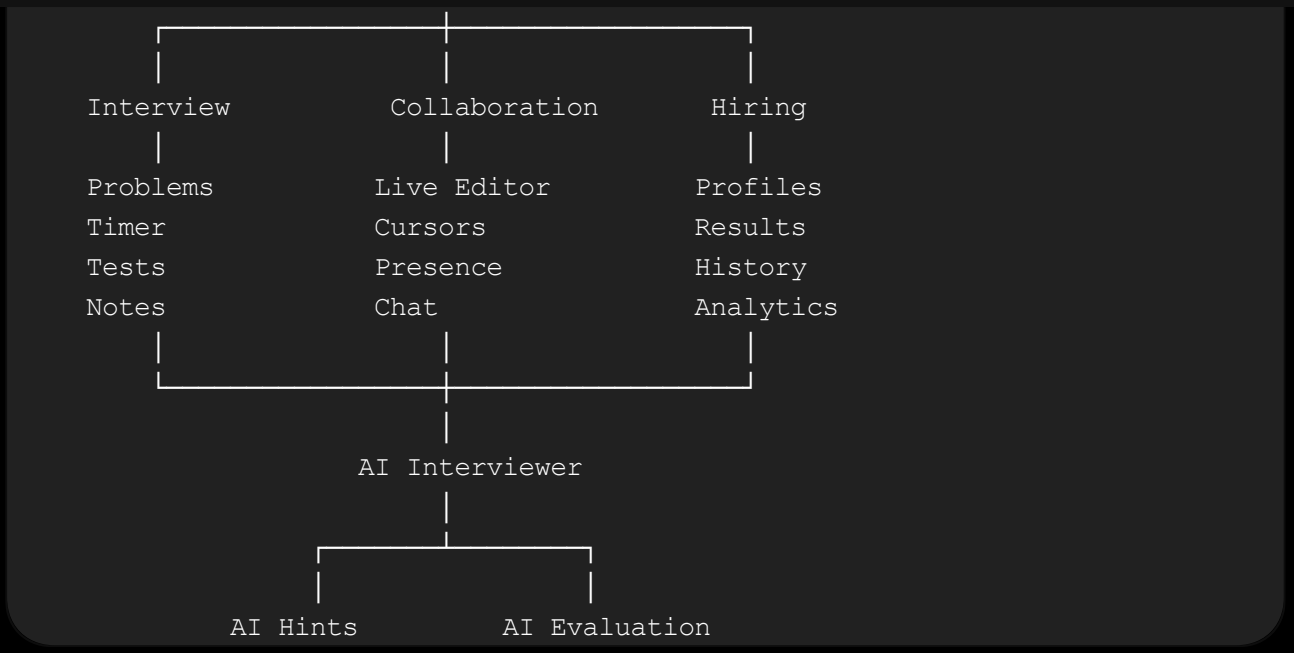
working software + deep CS knowledge.

---

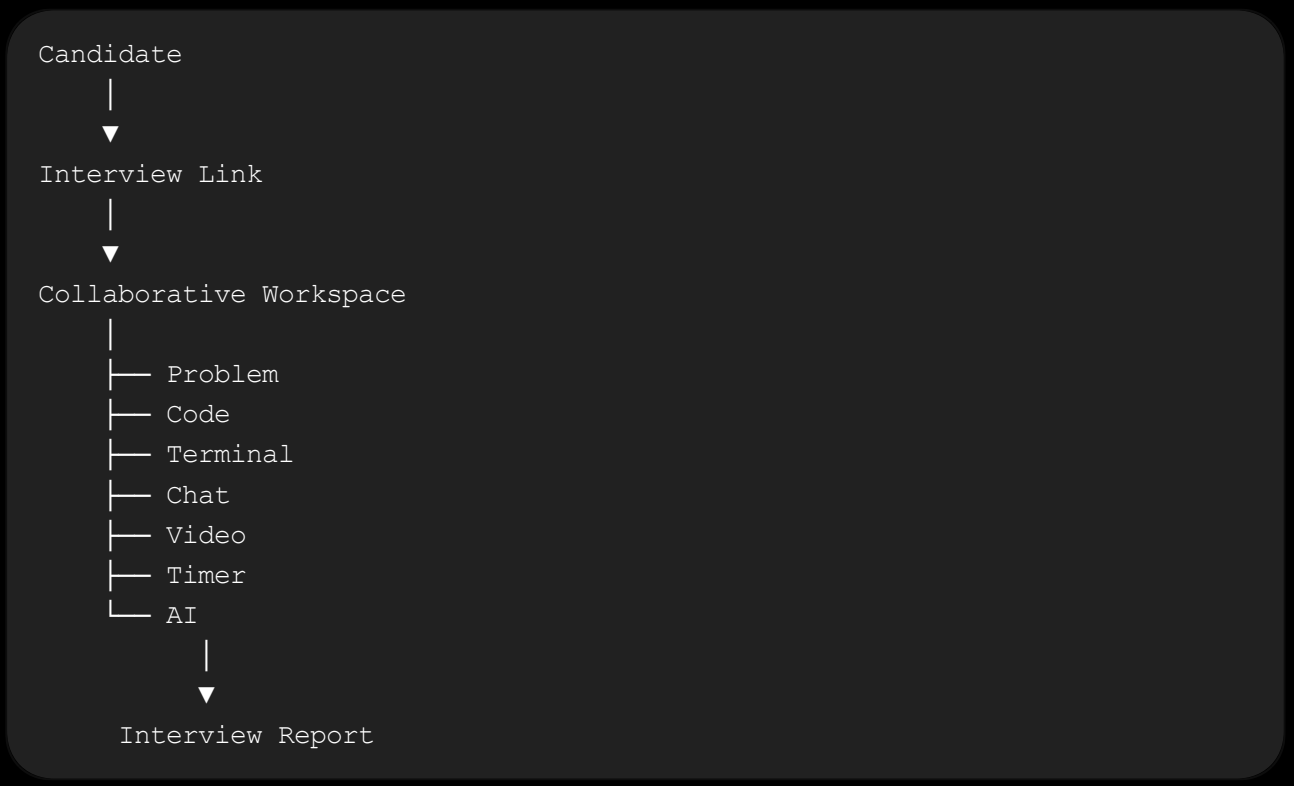
48. The ultimate version

If you keep developing this, the end product could become:

```
                    CODETOGETHER
                         |
```



And eventually:

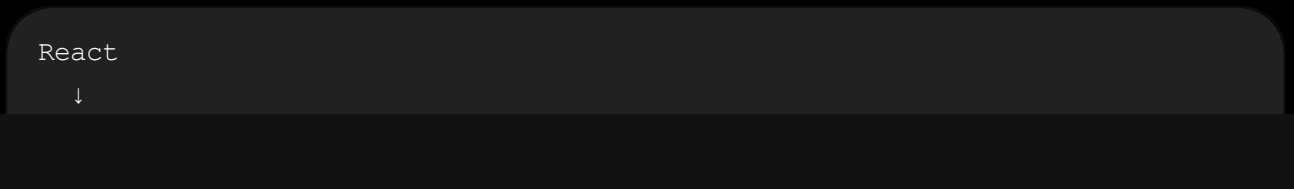


That is a **serious full-stack/distributed-systems project**, not just a college CRUD application.

If I were you, this is exactly how I'd start

Don't touch PostgreSQL, Docker, authentication, AI or WebRTC yet.

Build this first:



```
Monaco Editor
  ↓
Yjs
  ↓
WebSocket
  ↓
Node.js server
  ↓
Two browser tabs
```

Milestone 1

"I can open `/room/123` in two tabs and both tabs see each other's typing in real time."

Once that works, we'll add:

Milestone 2: live cursors + users
Milestone 3: problem panel
Milestone 4: interviewer/candidate roles
Milestone 5: code execution
Milestone 6: database + authentication
Milestone 7: interview history
Milestone 8: Docker sandbox
Milestone 9: video/audio
Milestone 10: AI interviewer

That order keeps the project manageable while making every milestone demonstrably useful.